

Service Function Chaining

Programmable Networks A.Y. 2024/25

Outline

- Introduction
- SFC Architecture
- Example of SFC implementation
- Orchestrating Service Function Chaining

Outline

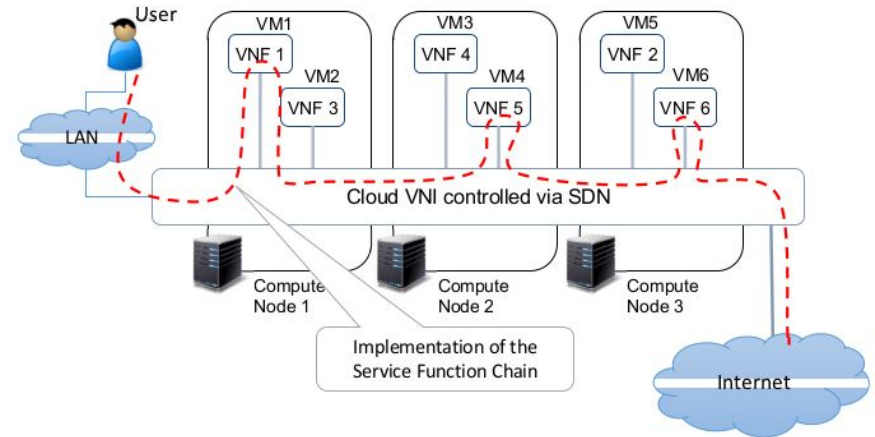
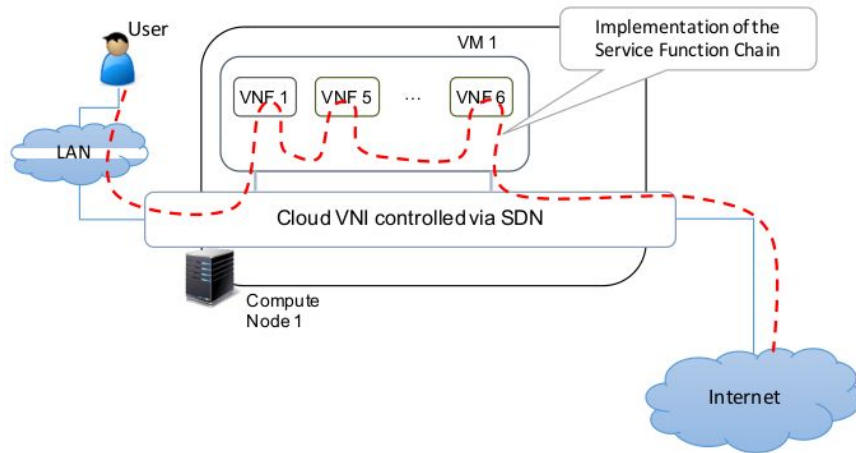
- Introduction
- SFC Architecture
- Example of SFC implementation
- Orchestrating Service Function Chaining

Introduction

- The coupling of NFV and SDN opens up the opportunity to re-think from scratch the way network services are designed
- Hardware components are replaced with Virtual Network Functions (VNFs) talking over NFV Infrastructures (NFVIs)
- Either deployment and connectivity acquire a degree of programmability never seen before
 - VNFs may be activated and de-activated almost on the fly
 - VNFs may be moved from a cloud infrastructure to another at will,
 - the connectivity among them may be re-designed on the fly by means of SDN

Service Function Chaining: what is?

- Service Function Chain (SFC) may be defined as the sequence of VNFs that a packet (or flow of packets) must traverse



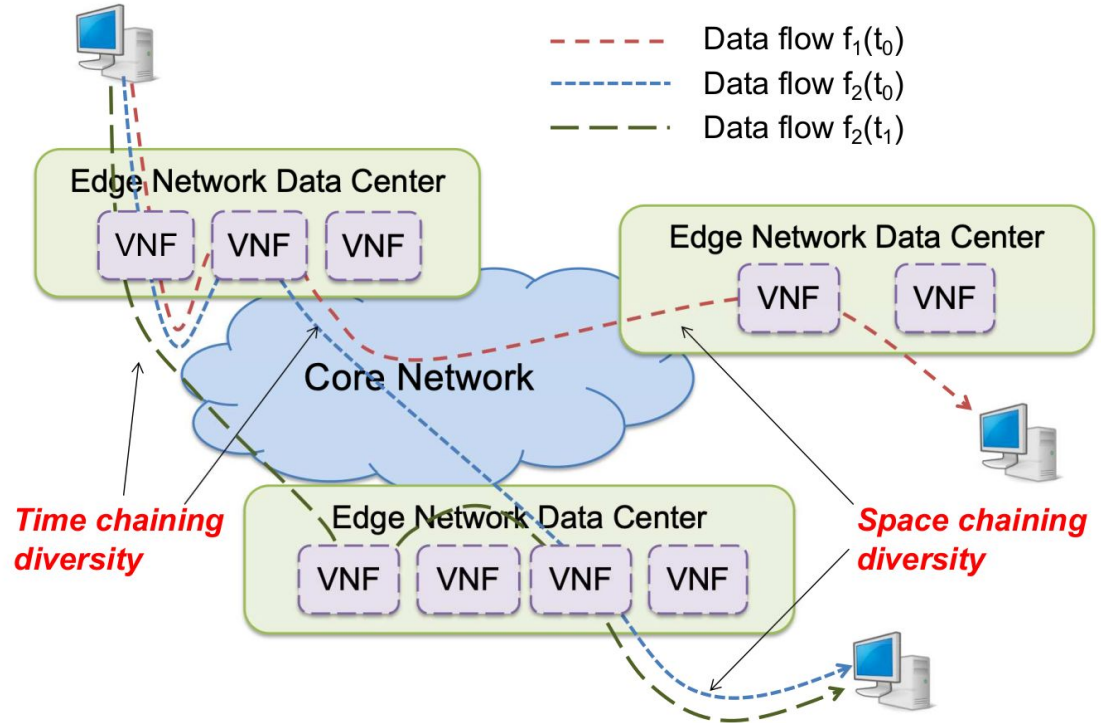
SFC: space and time diversity

Space diversity

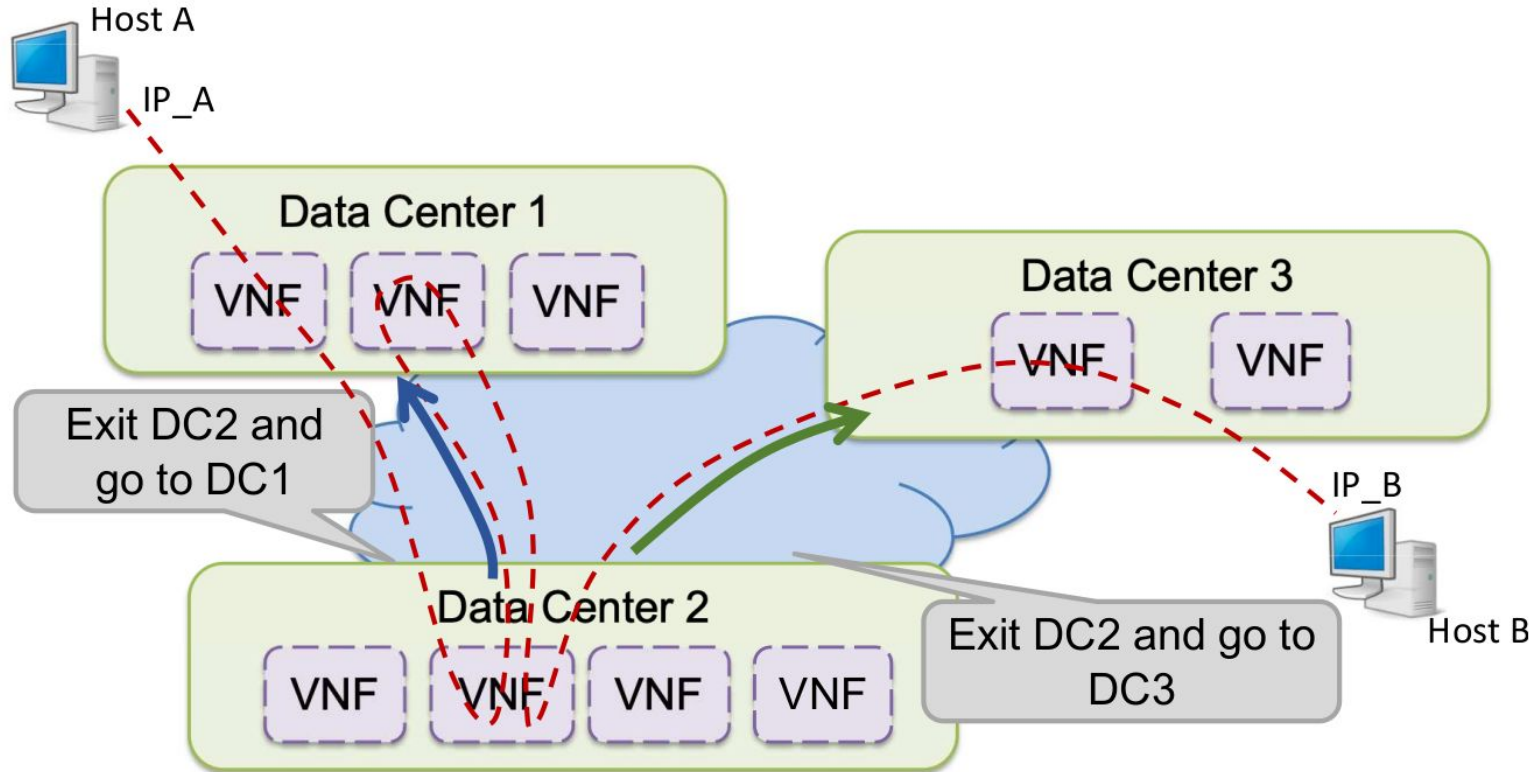
- flows $f_1(t_0)$ and $f_2(t_0)$ share the same VNFs in the edge data center at the source and then follow different paths in the core

Time diversity

- flow f_2 follows a given SFC at time t_0 and a different one at time $t_1 > t_0$



SFC is not Simple Forwarding



SFC is not Simple Forwarding

- SFC between Host_A with IP_A and Host_B with IP_B
- The packets flowing between the two hosts will carry IP_A and IP_B as either source or destination addresses
- In order to follow the SFC, a packet sent by Host_A should be
 - **routed back to Data Center 1** (blue arrow), after **the first time VNF2 is applied**
 - **forwarded to Data Center 3** (green arrow), after **the second time VNF2 is applied**
- Assuming classical IP forwarding, **it can generate conflicting situations**
 - e.g., in a router we need to use different output ports to reach the same destination
- **Packet forwarding in SFC must take into account the history!**
 - where to store it?

Outline

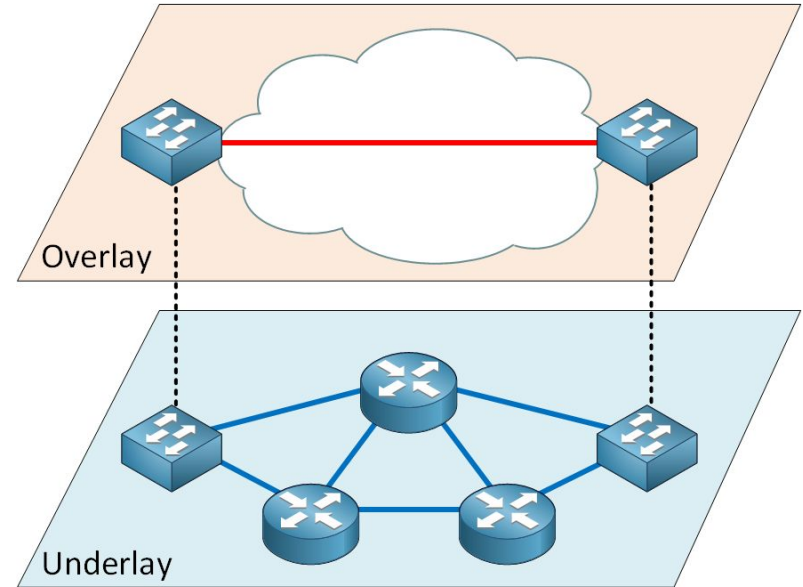
- Introduction
- **SFC Architecture**
- Example of SFC implementation
- Orchestrating Service Function Chaining

The SFC Architecture

- The Service Function Chaining architecture is described in the IETF RFC 7665
- An SFC specifies the **path that the packets should follow in the network**
 - This path is called the **Service Function Path** (SFP)
- The SFP a packet belongs to is an information stored in the packet itself, by means of a dedicated **encapsulation header** (a.k.a. SFC Header)
- SFC-aware network nodes and functions will exploit the SFP to realize Service Plane functionalities
 - **packet forwarding in the underlying network topology is a task of the network protocol**

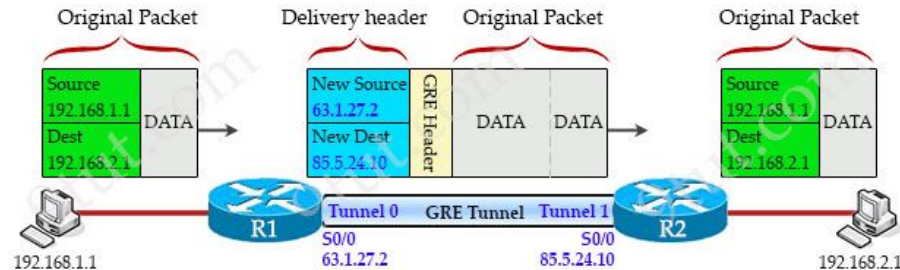
Underlay and Overlay

- SFC is an **overlay architecture** built on top of an underlay
- The underlay
 - **provides connectivity** to the overlay
 - is **unaware** of the existence of the overlay
 - it does not look inside the SFC Header
- Point to Point links in the overlay are multi hop paths in the underlay
 - SFC does not specify any technology to create these overlay links



Packet Tunneling

- Encapsulation of a layer x PDU in a layer y PDU (with $x \geq y$) is generally referred to as **packet tunneling**
 - GRE, IPSec, MPLS, VXLAN, etc.



- Tunneling techniques can be used to connect two SFFs
 - Intermediate nodes will process only the outer header

SFC Service Plane Logical Elements

- **SFC Classifiers**

- classifies the incoming traffic based on predefined policies
- it adds the SFC header with the correct SFP identifier

- **Service Functions**

- responsible for processing the packets along the chain
- it can either be SFC-aware (i.e., able to act on SFC-encapsulated packets) or SFC-unaware (i.e., it can only handle packets without SFC encapsulation)

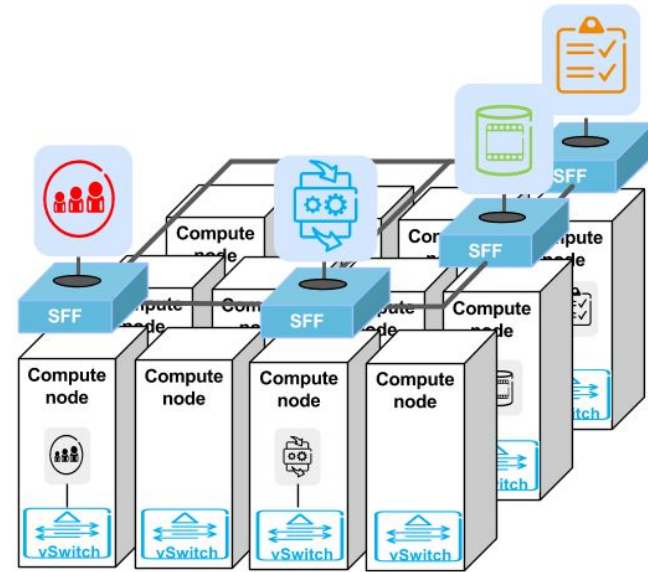
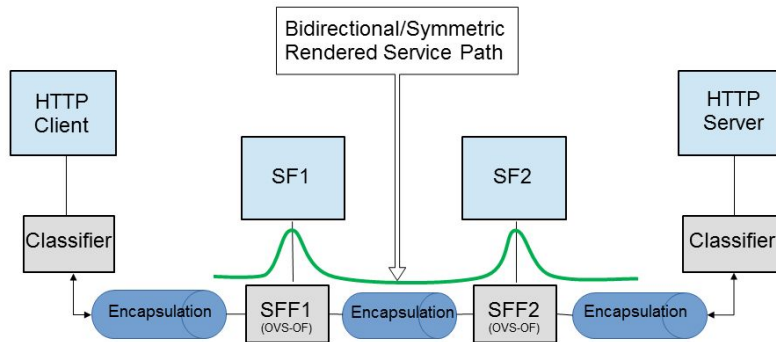
- **Service Function Forwarders (SFFs)**

- responsible for forwarding traffic to one or more connected Service Functions according to the information carried in the SFC header

- **SFC Proxies**

- responsible to remove and insert SFC encapsulation on behalf of SFC-unaware Service Functions

SFC Architecture scheme



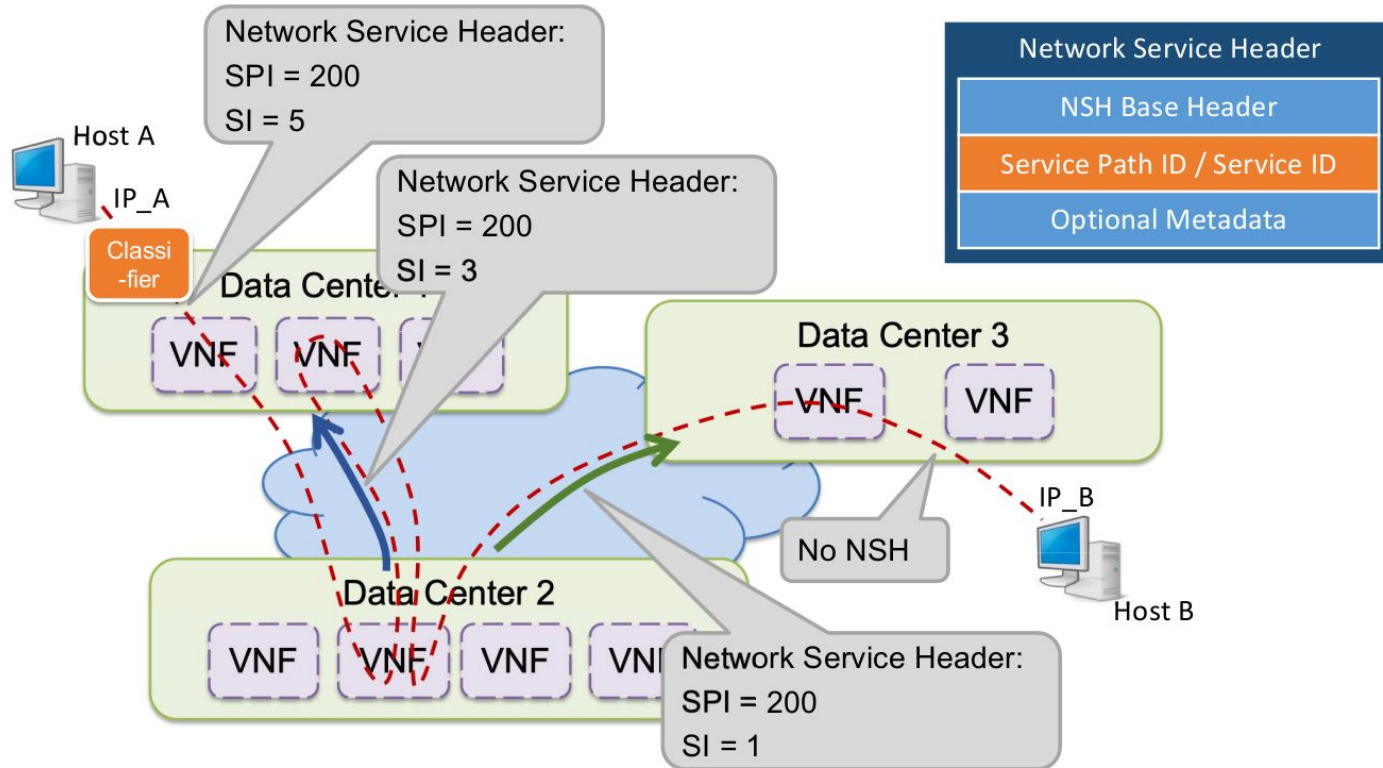
SFC Control Plane Interfaces

- **interface C1**
 - towards SFC Classifiers, used to manage SFC classification rules in classifiers
- **interface C2**
 - towards Service Function Forwarders, used for exchanging required information for SFC forwarding decision-making, collect state information on SFPs, etc.
- **interface C3**
 - towards SFC-aware Service Functions, used to collect output information resulting from the processing of packets in the Service Function
- **interface C4**
 - towards SFC Proxies, used to communicate SFC instructions and to retrieve state information

Network Service Header (NSH) as Service Plane Protocol

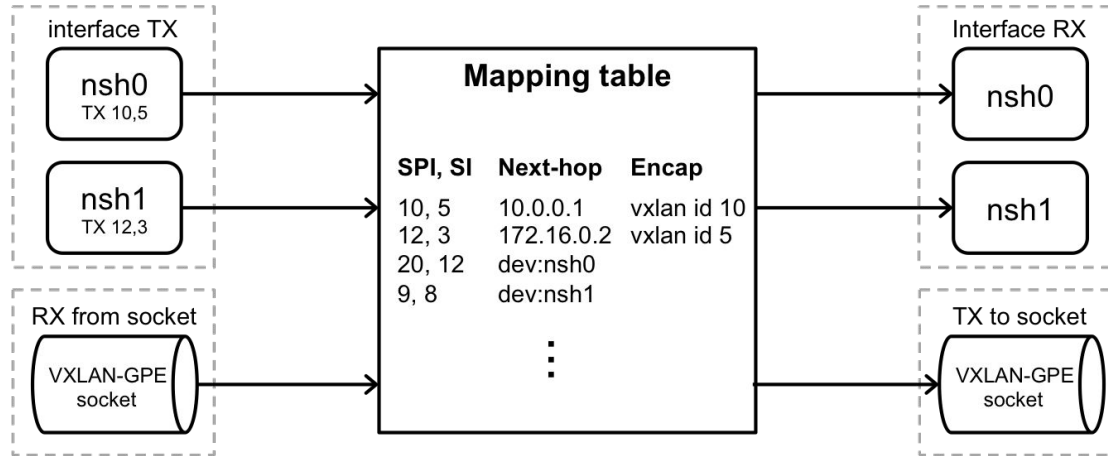
- NSH provides
 - SFP identification
 - transport-independent chaining
 - packet-based network and service metadata
- The two most important fields in the NSH header are the **Service Path Identifier (SPI)** and the **Service Index (SI)**
 - The SPI is a 24-bit integer number assigned to packets by the first SFC Classifier in the SFP
 - it is an **identifier of a SFP**
 - The SI is an 8-bit integer number used to identify the location within the SFP
 - it is **counter that is decremented by each node**

NSH: an example



NSH-to-transport Mapping Table

- NSH encapsulated packets go through look-up mapping table process
- Look-up key of the table is SPI/SI pair, and values of entries are next hop IP address and encapsulation type or nsh interface

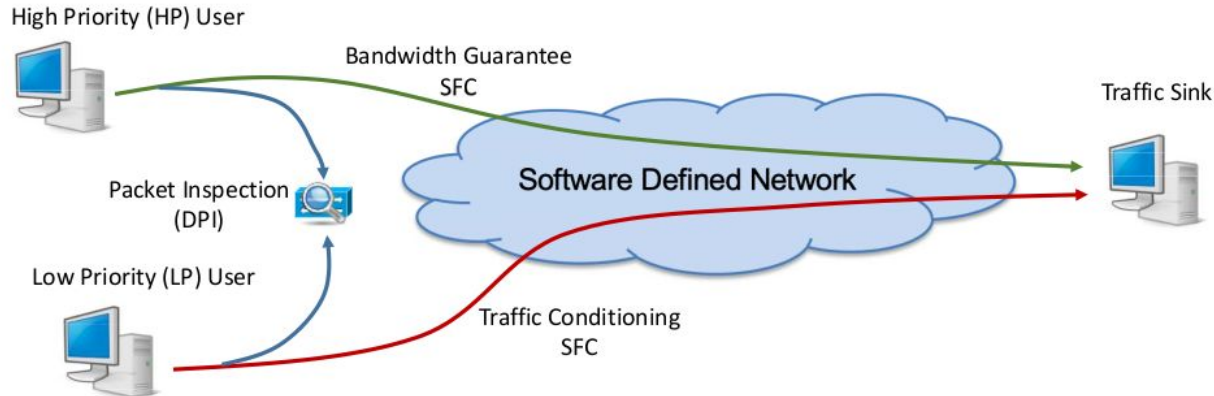


Outline

- Introduction
- SFC Architecture
- Example of SFC implementation
- Orchestrating Service Function Chaining

The Reference Scenario

- **QoS differentiation use case**, used to discuss the SFC implementation example
- A **High Priority (HP)** and a **Low Priority (LP)** user share a link and the network will react to overload by throttling the LP user to provide a guaranteed bandwidth to the HP user



Implementing SFC with NSH and OpenFlow

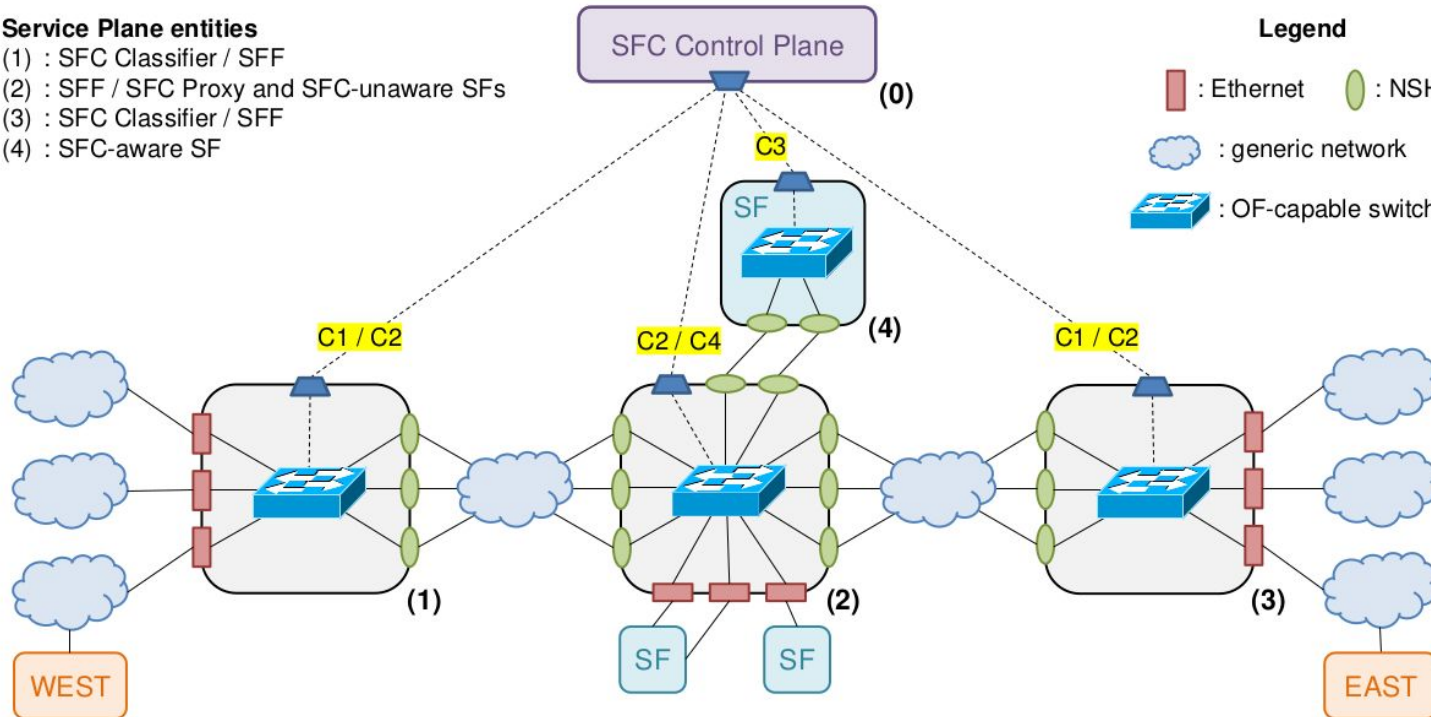
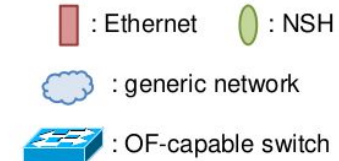
- **OpenFlow protocol** to implement the communication between the SFC Control Plane and Service Plane components and employing **NSH** for encapsulation
- Scenario composed of
 - SFC Control Plane entity
 - a pair of SFC Classifiers
 - an intermediate node serving as both SFF and SFC Proxy
 - a SFC-aware Service Function
 - two SFC-unaware Service Functions

Implementing SFC with NSH and OpenFlow

Service Plane entities

- (1) : SFC Classifier / SFF
- (2) : SFF / SFC Proxy and SFC-unaware SFs
- (3) : SFC Classifier / SFF
- (4) : SFC-aware SF

Legend



Practical Validation: basics

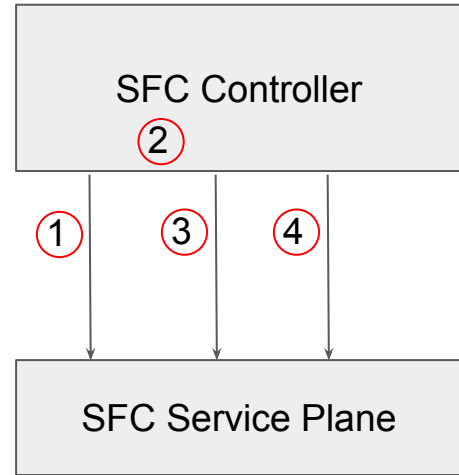
- The NSH capability is achieved via the **open-source NSH kernel module**, enabling the creation of NSH interfaces on each NSH node
- Each NSH interface
 - is **assigned** a **SPI/SI pair**
 - is **mapped** to a **transport-level next-hop** (i.e., an IP address), instructing the node to use VXLAN as encapsulation protocol
 - is **made aware of** the **inbound SPI/SI values** it is meant to receive
- Deep Packet Inspection (DPI), a Traffic Controller/Shaper (TC), and an Integrity Checker (IC) used as SFs

Practical Validation: SFC definition

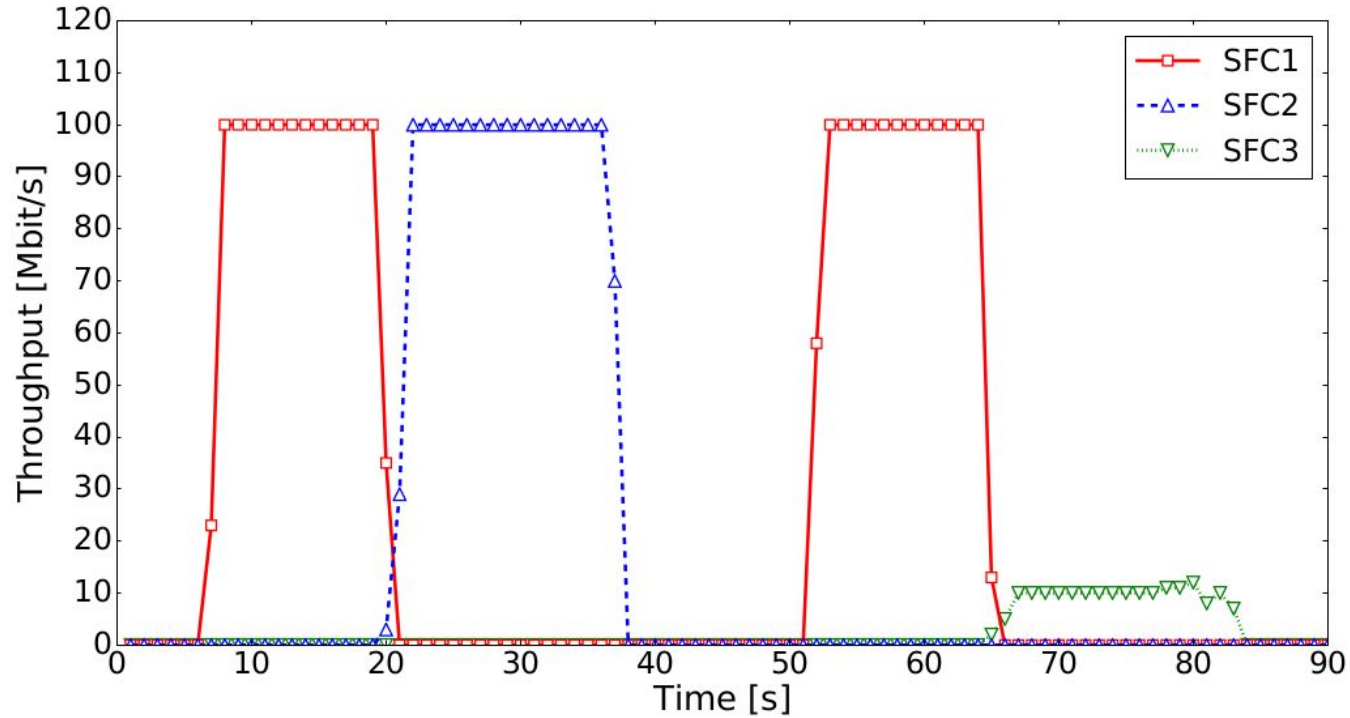
- **WEST** hosts are **clients**, while **EAST** hosts act as **servers**
- **Requirements**
 - All EAST-to-WEST traffic must be inspected
 - low priority traffic must be limited, so as to limit network occupancy
 - high priority traffic must be checked for integrity
- This can be accomplished by defining **three separate SFCs**:
 - SFC1, from any WEST user to the destination EAST user, duplicating the traffic towards the DPI
 - SFC2, from a **high-priority** WEST user to the destination EAST user, passing through the IC
 - SFC3, from a **low-priority** WEST user to the destination EAST user, passing through TC for bandwidth limitation

Practical Validation: Dynamic SFC

1. install proactive flow rules in the switches so as to apply chain SFC1
2. wait for any WEST to EAST flow to start
3. when the flow starts, the orchestrator starts the DPI, and after a small time period, it retrieves information from it
4. high priority flow?
 - a. yes - install rules applying SFC2
 - b. no - install rules applying SFC3



Practical Validation



throughput measured at node 2

Outline

- Introduction
- SFC Architecture
- Example of SFC implementation
- Orchestrating Service Function Chaining

High level orchestration with OSM

- Open Source Mano allows the definition of SFC in the form standardized in the ETSI-MANO framework
- This **information can be added inside the Network Service Descriptor (NSD)** yaml file
- A **VNFFG Descriptor (vnffgd)** element is composed by **two main lists**
 - the **Rendered Service Path** (rsp)
 - defines all paths belonging to a VNFFG
 - each path is a list of VNFs that need to be traversed, each one described by the tuple
 - <VNF, order, entry interface, exit interface>
 - the **Classifier** (classifier)
 - contains the details regarding the type of traffic
 - defined as a series of match attributes (match-attributes)

High level orchestration with OSM

```
nsd:nsd-catalog:
```

```
  nsd:
```

```
    ...
```

```
    vnffgd:
```

```
      - name: VNFFGD_NAME
```

vnffgd

rsp

```
      rsp:
```

path

```
        - name: RSP_NAME
```

```
        vnfd-connection-point-ref:
```

tuple

```
          - vnfd-ingress-connection-point-ref: VNF_INGRESS_CP
```

```
            vnfd-egress-connection-point-ref: VNF_EGRESS_CP
```

```
            vnfd-id-ref: VNF_NAME
```

```
            order: INT
```

```
          ...
```

```
        ...
```

classifier

```
        classifier:
```

match-attribute

```
          - rsp-id-ref: RSP_NAME
```

```
            match-attributes: TRAFFIC_TYPE
```

SFC in OpenStack

- SFC extension provides in OpenStack an API to support SFP creation and deployment in Neutron
- The SFC extension defines and deploy a SFP according to a **four-step approach**
 - **instantiation of a Flow Classifier (FC)** to classify the incoming traffic based on predefined policies (e.g., header matching rules)
 - **Port Pairs (PPs) are created**
 - PP represents a service function instance that includes an ingress and egress port
 - **creation of Port Pair Groups (PPGs)**, which are a collection of one or more port pairs (useful for load balance the traffic over multiple instances of an SF)
 - **Port Chain (PC) is instantiated** as a binding between one or more FCs and an ordered list of PPGs

SFC in OpenStack

